An exact integer pre-filter for the NumberFields@home discriminant test

The Passiflora polDisc pre-filter: what it computes, why it is correct, and how that was checked

27 October 2026

Abstract

NumberFields@home tests very many monic degree-10 integer polynomials with an exact discriminant test that dominates the running time of the CPU application. Passiflora puts a pre-filter in front of it: a polynomial is dropped when, for one of a few primes q, the Legendre symbol of its discriminant modulo q is -1, which proves that it cannot pass. The pre-filter must never drop a polynomial that passes, or the project loses results. The first version computed the symbols in floating point and was exact only under round-to-nearest without x87 code; it lost valid polynomials when either failed. The present version uses only integer arithmetic (signed Montgomery reduction on the CPU, unsigned Montgomery reduction on the GPU). This paper states the algorithms, the theorems that make the decisions exact (the program text computes exactly the decisions of the exact algorithms, independently of the floating-point environment), the contract between the filter and the application, and the evidence: a test of the compiled filter on 25.5 billion polynomials, complete sets at tiny primes, an exact oracle with deliberate faults, a differential test of the exact test of the original GetDecics application, the 31 test work units in every build, and a shim of the GPU kernel. Along the way it shows that this exact test itself silently drops passing polynomials when its first `int64` steps overflow, a defect of GetDecics that Passiflora neither causes nor changes. The proofs are in the full document [1]; here every result is stated with its status and with what it still assumes.

Contents

1 The problem **1**

2 The two chains and their theorems **2**

3 What the exact test of GetDecics computes **3**

4 Exact integer arithmetic **5**

5 The GPU filter **6**

6 The contract with the application **6**

7 The evidence **7**

8 What it all rests on **8**

1 The problem

NumberFields@home searches for number fields of degree 10 by enumerating monic polynomials $f = x^{10} + a_9x^9 + \cdots + a_0$ with integer coefficients.

GetDecics and Passiflora. The program that volunteers run for this is *GetDecics*, the project's original application [2]. Passiflora is not a new program: it is a patch on top of the original GetDecics application (source at commit `638ca4e`), plus new source files for the pre-filter and the GPU code. The enumeration and the exact discriminant test are GetDecics' own code; Passiflora decides which polynomials reach that test, and its output must be exactly that of the original application. In the full document [1] and in the repository, the original GetDecics application is called "upstream".

The final test of the enumeration is the *discriminant test* (`polDiscTest_cpu` of GetDecics, built on GMP):

Definition 1 (Passing)**.** Let S be a finite set of primes (read from the work unit; $S = \{2, 5\}$ in every test work unit). A monic $f \in \mathbb{Z}[x]$ *passes* if $D = \operatorname{disc}(f) \neq 0$ and $|D| = \big(\prod_{p \in S} p^{\alpha_p}\big) m^2$ for some integers $\alpha_p \geq 0$, $m \geq 1$.

Almost no polynomial passes, and the test is expensive (the repository reports that the stock CPU application spends 98% of its time in it [reported, not re-checked]). The pre-filter rests on one observation.

Proposition 2 (The rejection rule)**.** *[proved] Let f pass, and let q be a prime with $q \equiv 1 \pmod 4$ at which every $p \in S$ is a nonzero square. Then $\left(\frac{\operatorname{disc} f}{q}\right) \in \{0, 1\}$.*

Indeed $\chi(D) = \chi(\pm 1) \prod_p \chi(p)^{\alpha_p} \chi(m)^2$, $\chi(-1) = 1$ because $q \equiv 1 \pmod 4$, and $\chi(p) = 1$ by the choice of q. So a symbol -1 at one such prime is a proof that f fails. The CPU filter uses 30 such primes: six *small* ones below 2^{10} $(1009, 929, 881, 809, 769, 761$ for $S = \{2, 5\})$ and 24 *large* ones below 2^{25} $(33554249, \ldots, 33551849)$. Each rejects about half of the candidates; survivors of all 30 go on to the exact test, which decides them as before. The GPU kernel applies the same filter at up to 24 primes in $(256, 1024)$ (14 exist for $S = \{2, 5\}$) before it hands the survivors to the CPU.

Why integer arithmetic. The first version of the filter (commit `42c3270`) stored residues in floating point and reduced with the rounding trick $x - q \cdot ((x\,q^{-1} + M) - M)$, which is exact only if every operation is rounded to nearest in the declared precision. The fault guide of [1, Section 14] reproduces, with the real application, what happens otherwise [checked numerically]: built for x87 arithmetic (as a 32-bit x86 build is), the application lost valid polynomials in four of the seven smallest test work units (6, 12, 6 and 5), among them the only real result of one unit; a library that switches the rounding mode to "up" before the filter is initialised makes it reject valid polynomials; rounding down or toward zero makes it reject nothing. A filter whose correctness depends on the floating-point environment of the volunteer's machine cannot be proved correct for the program that ships. The integer version removes the dependence.

2 The two chains and their theorems

Everything here is over the field $\mathbb{F}_q$, q an odd prime with $q \nmid 10$; χ is the quadratic character. For monic f of degree n, $\operatorname{disc} f = (-1)^{n(n-1)/2} \operatorname{Res}(f, f')$, and the discriminant of f modulo q is the residue of $\operatorname{disc} f$ [proved]. The filter computes the symbol of $\operatorname{Res}(f, f')$ by Euclid's algorithm on f and f', without inverses: instead of dividing by a leading coefficient it multiplies by it (pseudo-division), and accounts for the powers it introduces. Only the parity of their exponents matters for a Legendre symbol [1, Section 4].

`chain_block`, the fast chain. For a *normal* remainder sequence of (f, f') (degrees $n, n-1, \ldots, 1, 0$) every step is two rounds with the same multiplier $\lambda = \operatorname{lc}(B)$: $R = \lambda A - \tau x B$ and $P = \lambda R - tB$, which equals $\lambda^2 (A \bmod B)$ (Lemma L2). The squares λ^2 never change the symbol, so the final constant has the symbol of the discriminant. The chain runs 64 polynomials in the same instructions (SIMD lanes) and checks, at each step, that the next leading coefficient is nonzero.

Theorem A (`chain_block` and its check)**.** *[proved] Let f be monic of degree $n \geq 2$ over $\mathbb{F}_q$, $q \nmid n$. The check passes at every step if and only if the remainder sequence of (f, f') is normal, and the first failing check is at the first abnormal degree. If it passes, the final constant `base` satisfies $\chi(\texttt{base}) = \chi\big((-1)^{n(n-1)/2}\big)\chi(\operatorname{disc} f)$; for $q \equiv 1 \pmod 4$ this is $\chi(\operatorname{disc} f)$, and a rejection means $\chi(\operatorname{disc} f) = -1$.*

`chain_general`, the general chain. Polynomials are stored left-aligned, so that one elementary step $W = l_v U - l_u x^{u-v} V$ is the same formula in every lane whatever the degrees; leading zeros of W are shifted out, U and V are swapped when $\deg W < \deg V$, and an accumulator `mm` collects l_v exactly at the steps whose exponent $e = \deg W - u + v$ is odd (by $\operatorname{Res}(V, W) = l_v^e \operatorname{Res}(V, U)$). It follows any degree sequence.

Theorem B (`chain_general`)**.** *[proved] Under the same hypotheses the algorithm stops within $2n - 2$ iterations (18 for $n = 10$; the code allows 40). Either a remainder is 0, and then $\operatorname{disc} f \equiv 0$ and the polynomial is kept; or it ends with a nonzero constant, and then $\chi(\texttt{mm}) = \chi\big((-1)^{n(n-1)/2}\varepsilon\big)\chi(\operatorname{disc} f)$ with a sign ε from the swaps. For $q \equiv 1 \pmod 4$ this is $\chi(\operatorname{disc} f)$, and a rejection means $\chi(\operatorname{disc} f) = -1$.*

The condition $q \equiv 1 \pmod 4$ is not decoration: with $q \equiv 3 \pmod 4$ the signs of the swaps make `chain_general` wrong in about a quarter of the degree-4 and 70% of the degree-10 cases tested [checked numerically].

The filter. For each prime in turn, every surviving polynomial goes through `chain_block`; the lanes whose check fails are re-run through `chain_general` (pass 2). A family of polynomials that keeps failing the check is sent to `chain_general` directly (`prefgen`), re-probed every 64 calls. The routing changes the speed, never a decision:

Theorem C (soundness and exactness)**.** *[proved] In exact arithmetic over $\mathbb{F}_q$: for every input, block size and routing state, `pdf_filter` gives a polynomial the flag 0 only if it is monic and has $\left(\frac{\operatorname{disc} f}{q}\right) = -1$ at some filter prime; so it never rejects a polynomial that passes. Conversely it rejects a monic polynomial exactly when some filter prime has symbol -1, at the first such prime.*

The converse is what makes a test with an exact oracle possible (Section 7).

3 What the exact test of GetDecics computes

The filter is proved against the definition of passing, but in the application its rejections take the place of verdicts of the original GetDecics application's own exact test, `polDiscTest_cpu`. So it matters what that test computes. It follows Cohen's sub-resultant algorithm [3, Algorithm 3.3.7]: starting from f and f' it computes a chain of pseudo-remainders whose last element gives the resultant $\operatorname{Res}(f, f')$, and so the discriminant, up to known factors. The numbers in that chain grow to about a thousand bits, so the test uses GMP; but to save time it does the first two iterations, where the numbers are still small, in 64-bit machine integers (`int64`), and only then switches to GMP. At the end it removes every factor of the primes of S from its result `polDisc` and asks whether what remains is a perfect square.

Proposition 3 (What the exact test computes)**.** *[proved] Let f be monic of degree 10 with `int64` coefficients, and suppose (H): no `int64` value computed in the first two iterations of `polDiscTest_cpu` leaves the `int64` range. Then $|\operatorname{disc} f| = r^2\,|\texttt{polDisc}|$ for a nonzero rational r, and `polDiscTest_cpu` sets the flag to 1 exactly when f passes.*

The proof [1, Proposition 3.19] follows the code of GetDecics step by step against Cohen's algorithm (whose exact divisions are quoted [standard result, quoted]): the content of f' divides 10 and is found exactly, every pseudo-remainder is the unique one by uniqueness of division in $\mathbb{Q}[x]$, the last step's shortcut replaces the resultant by an even power of a rational, and the removal of S is complete.

The overflow. Hypothesis (H) is not a formality. The two `int64` iterations multiply coefficients of f by coefficients of f', and then by coefficients of the first remainder, and nothing checks that these products fit into 63 bits. The source of GetDecics notes the danger for one branch of its second iteration ("the result would exceed a long") and sends that branch to GMP, but the two branches it keeps in `int64` are not bounded either. A signed overflow is undefined behaviour in C; in practice the value wraps around modulo 2^{64}, the GMP steps that follow work on a wrong number, the result is almost never a perfect square, and the test answers "does not pass". A polynomial that passes is then lost silently: no error, no message, and the work unit's output simply lacks it.

How it was measured. The program `test_poldisc.cpp` links the compiled `polDiscTest_cpu` of GetDecics and compares its flag with the definition computed independently by PARI (the discriminant, the primes of S removed, the square test). It also replays the `int64` steps of GetDecics in 128-bit integers, which shows whether one of them overflows. It runs eight families of polynomials, among them the real candidates of the application and polynomials built to pass:

What it shows [checked numerically]. Without overflow GetDecics agrees with the definition on all 8,891,443 polynomials, as the proposition says. With overflow it does not: of a million polynomials built to pass with large coefficients ($G(x^2)$ shifted, $|G_k| \le 10^6$), 844,277 overflow and GetDecics rejects 844,265 of them although every one passes. For random coefficients the overflow starts between sizes 1000 and

Table 1: The compiled `polDiscTest_cpu` of GetDecics against the definition (PARI), and the Passiflora filter, on 10,735,681 polynomials ($S = \{2, 5\}$). "ovf": an `int64` step of GetDecics overflows; "differ": the flag of GetDecics is not the definition's.

		pass			differ		filter	
family	polynomials	GetDecics	definition	ovf	no ovf	ovf	rejects	rejects passing
random, $\lvert a_i\rvert \le 3$	3,000,000	458	458	0	0	0	2,930,945	0
random, $\lvert a_i\rvert \le 100$	2,000,000	0	0	0	0	0	1,999,948	0
random, $\lvert a_i\rvert \le 10^6$	1,000,000	0	0	999,961	0	0	1,000,000	0
built to pass, small	2,000,000	1,999,939	1,999,939	0	0	0	0	0
built to pass, large	1,000,000	155,735	1,000,000	844,277	0	844,265	0	0
square factor	1,000,000	0	0	0	0	0	0	0
real candidates	733,855	0	0	0	0	0	733,855	0
the 83 results, transformed	1,826	1,826	1,826	0	0	0	0	0
all	10,735,681	2,157,958	3,002,223	1,844,238	0	844,265	6,664,748	0

3000: in samples of 20,000, no overflow with coefficients up to 1000, 43% of the polynomials at 3000, 87% at 10^4, almost all from 10^5. Coefficient size alone is not the measure, though: the real candidates have coefficients up to $2.1 \cdot 10^9$ and never overflow, because their coefficients grow with the degree in the way that polynomials with roots of moderate size do. What matters is the largest value the `int64` steps form.

An example from a real result. One of the 83 results in the truth files of the test work units is

$$f = x^{10} + 600x^7 + 1350x^6 - 800x^5 + 115000x^4 + 405000x^3 + 265625x^2 - 540000x + 185000.$$

Multiplying its roots by 3 gives $g(x) = 3^{10} f(x/3)$,

$$g = x^{10} + 16200x^7 + 109350x^6 - 194400x^5 + 83835000x^4 + 885735000x^3 + 1742765625x^2 - 10628820000x + 10924065000.$$

It defines the same number field, and $\operatorname{disc} g = 3^{90} \operatorname{disc} f$, so g passes as f does (PARI: `poldisc`, `issquare`, `nfisisom`). GetDecics rejects g: one of its `int64` values would be about 2^{67}. With the roots multiplied by 2 instead, the largest value is about $2^{56.5}$ and GetDecics accepts the polynomial.

Does it affect the project? On the 146,771 candidates captured from seven real work units the largest `int64` value is $2^{59.45}$ (none above 2^{60}), a factor of about 12 below the limit 2^{63}; on the 83 results it is $2^{48.5}$. So no result of the sampled work units is lost, but the margin is three and a half bits, measured on samples, not proved, and the example shows that a passing polynomial whose roots are only three times those of a real result, defining the same field, is already lost. Whether a work unit with larger coefficients loses results is not known; (H) remains a named assumption of this paper.

Passiflora and the overflow. The defect is in GetDecics and is present in the stock application, with or without Passiflora. Passiflora keeps the exact test as GetDecics has it (its patch changes only the test's conversions for a 32-bit `long`), so that its output stays the stock program's. The filter rejects only polynomials that provably do not pass, by the definition; a passing polynomial that the overflow makes GetDecics lose is kept by the filter and then lost by the exact test exactly as in the stock program. The filter could differ from the stock program only by rejecting a polynomial that GetDecics wrongly *accepts* because of an overflow, which would make the output more correct; it did not happen in the tests (last column of the table). The fix belongs to GetDecics: do the two `int64` iterations with overflow-checked arithmetic (GCC's and Clang's `__builtin_mul_overflow` and `__builtin_sub_overflow`) or in 128-bit integers, and compute those iterations with GMP when a value does not fit, at the cost of a few instructions per product.

4 Exact integer arithmetic

The representation. Elements of $\mathbb{F}_q$ are integers in a small range around 0. Products are reduced by signed Montgomery reduction: for $R = 2^k$, $qq' \equiv 1 \pmod R$ and $m = s_k(xq')$ (the signed residue in $[-R/2, R/2)$), $u = (x - mq)/R$ is an integer, $u \equiv xR^{-1} \pmod q$ and $|u| \le |x|/R + q/2$ [proved]. The small primes use $R = 2^{16}$ in 16-bit lanes (`mont`, 16 lanes per AVX2 register, compiled to `vpmulhw`/`vpmullw`), the large ones $R = 2^{32}$ with 64-bit products (`sredc`). A 64-bit coefficient is split once into three 21-bit pieces and loaded at each prime by one reduction (`ldres`).

What the language must do. The proofs use three rules: (S1) an operation whose exact result fits its kind gives that result; (S2) a conversion to a narrower kind reduces modulo 2^k; (S3) `shifta` is an arithmetic shift and `iand` with $2^k - 1$ keeps the low bits. The Fortran standard leaves (S2) and the bit model to the processor; the GNU Fortran manual documents (S3) but says nothing about (S2), which GCC documents for C. Standard-conforming replacements exist but made `pdf_filter` 2 to 6 times slower in a micro-benchmark (AVX2: 105–126 ns per polynomial as written, 572–697 with `modulo` and exact division, 225–262 with offsets) [checked numerically], because the compiler then no longer uses the 16-bit multiply-high instructions. So the code keeps them, and they are checked in three ways: the compiled reductions against exact models on all inputs that matter (GCC 15.2 and 16.2), the whole filter on complete sets at tiny primes (Section 7), and at every start of the program (below).

Theorem D (the integer program is exact)**.** *[proved] Assume (S1)–(S3), and let the primes be those that `pdf_init` chooses. Then for every input buffer, block size and routing state, at every stage and in every lane, the code's decisions (the check of `chain_block`, `keep`) are those of the exact algorithms over $\mathbb{F}_q$ on $f \bmod q$. So Theorem C holds for the program as written. The filter executes no floating-point operation; its result depends neither on the rounding mode nor on x87 or SSE code nor on `-ffast-math`.*

The proof [1, Section 8] has four parts.

1. *Reductions and loading.* `mont` returns a value in $(-q, q)$ for operands in $(-2q, 2q)$ when $q \le 2^{13}$, `sredc` for $|x| < 2^{31}q$, and the three-piece loading gives $c \bmod q$ in $(-q, q)$ for every `int64` coefficient c; no operation overflows.
2. *Ranges.* Every value of the four chains stays where those lemmas apply: small-prime values in $(-2q, 2q)$ (a value is 0 mod q exactly when it is 0 or $\pm q$), large-prime values in $(-q, q)$.
3. *Weights.* Each polynomial the code holds is its exact counterpart times a power of R^{-1}, the same for all its coefficients, because both terms of every difference carry the same power. So every zero test, degree and swap agrees with the exact run, and as $R = 2^{16} = (2^8)^2$ and $2^{32} = (2^{16})^2$ are squares, no Legendre symbol changes. Euler's criterion runs in the Montgomery domain from $R \bmod q$, and -1 is recognised as the residue of $-R$.
4. *The masked loop.* `chain_general` runs all 64 lanes in one loop, masking the lanes that have finished and using a cheaper "lockstep" branch when all active lanes have the same degrees. A lemma [1, Lemma 8.10] proves, by induction over the iterations, that every lane computes the one-lane algorithm: entries beyond a degree are zero, a masked lane is never changed, the lockstep branch runs only when every lane is active with the same degrees, and copying the first $\max\deg + 1$ entries copies whole polynomials.

The size limits, and the guard. The single-reduction arguments stop at $q \le 2^{13}$ (small) and $q \le 2^{30} + 1$ (large). Following the bounds of the values through the chains instead gives larger limits for the whole filter, proved with a complete finite computation [checked numerically]: small primes up to 8385 and large primes up to 1,097,010,265. This holds although `mont` itself has counterexamples from 8241 and `sredc` on a difference from $2^{30} + 23279$, because the chains never form those operands. The limits are not known to be sharp. The shipped primes are far inside them, below 2^{10} and 2^{25}. `pdf_init` enforces them: a usable prime outside $[11, 8385]$ (small) or $[11, 1{,}097{,}010{,}265]$ (large) is skipped with one line on standard error, so no future change of the search can leave the proved region silently.

The start-up check. At the end of `pdf_init` the function `arith_ok` runs the compiled arithmetic on known inputs: `mont` and `sredc` on 64 inputs per prime including the extremes, the inverse relations $qq' \equiv 1$, the split of a coefficient into pieces, and the loading, all compared with values computed by

`modulo` and exact division only, which do not depend on (S2) or (S3). If it fails, `pdf_init` writes one line on standard error and chooses no prime, so that every flag is 1 and the exact test alone decides [proved]. It guards against a compiler or flag set without the behaviour of (S2), (S3) on the machine where the program runs. It does not show that the code inlined into the chains was compiled the same way, and it evaluates its own copy of the coefficient split rather than `pdf_filter`'s lines; the deliberate fault "top piece shifted logically" in `pdf_filter` passes it (the oracle test sees that fault).

Instruction sets. The application contains an AVX2 build and a baseline (SSE2) build of the filter and picks the AVX2 one when the CPU reports AVX2 and FMA. The AVX2 build was compiled with `-march=x86-64-v3`, which also lets the compiler use BMI1/2, LZCNT, MOVBE and F16C, none of which the run-time test checks; it is now compiled with `-march=x86-64 -mavx2 -mno-popcnt`. The generated code is the same as before (114 distinct instructions, the same set as the `x86-64-v3` object, none outside AVX and AVX2 besides the base set) [checked numerically]; the change only makes the guarantee hold on paper as well.

5 The GPU filter

The GPU kernel runs the enumeration's innermost loops and, for every polynomial it counts, the one-lane `chain_general` at the small primes of its table. A polynomial it rejects is never shown to the CPU, so the kernel's filter must be sound by itself; survivors go through the CPU filter and the exact test as before. The earlier kernel used the single-precision floating-point trick. The present one uses unsigned 32-bit Montgomery arithmetic with $R = 2^{16}$, residues kept in $[0, 2q)$ (zero modulo q exactly when 0 or q), which needs no multiply-high instruction and no comparison per reduction.

Lemma 4 (`redc`)**.** *[proved] Let $q < 8192$ be odd, $q^- \equiv -q^{-1} \pmod{2^{32}}$, $0 \le t < 8q^2$, and $m = tq^- \bmod 2^{16}$. Then $t + mq < 2^{30}$ is divisible by 2^{16}, and $r = (t + mq)/2^{16} \equiv t\,2^{-16} \pmod{q}$ with $0 \le r < 2q$.*

Theorem E (the GPU filter is exact)**.** *[proved] For a prime $q \equiv 1 \pmod 4$ with $11 \le q < 8192$, the table row of q, and any `int64` coefficients $c_1, \dots, c_{10}$, the kernel's `survives` rejects $f = x^{10} + c_1 x^9 + \cdots + c_{10}$ at q exactly when $\left(\frac{\operatorname{disc} f}{q}\right) = -1$. The table's primes are usable for S, so a polynomial the kernel rejects does not pass.*

The proof [1, Section 8.10] bounds every value passed to `redc` below $8q^2$, follows the weights as on the CPU, and runs Euler's criterion in the Montgomery domain. It uses only the semantics of unsigned integers in OpenCL C and CUDA, which wrap modulo 2^{32} by definition. A first integer version, with $R = 2^{32}$, canonical residues and `mul_hi`, was also exact but made the kernel 47% slower; the version above is about 10% faster than the floating-point kernel (Section 7).

Host and device code must match. The theorem needs the kernel to read the table the program builds. For OpenCL this holds by construction (the kernel source is part of the program). The CUDA kernel is a fat binary embedded at build time and the MUSA kernel a library loaded at run time; a left-over binary of the floating-point kernel would read the new table wrongly and could reject passing polynomials, and the GPU self-check (which compares polynomial counts taken before the filter) would not notice. So the table layout has a number (format 2), the kernel's entry point carries it in its name (`nf_enum_t2`), and the MUSA library reports it and checks that every table row has the integer layout. A CUDA program with an old fat binary stops at start-up with a message; a MUSA program refuses an old library and runs on the CPU; an old program with a new library stops [checked numerically]. After any change of the kernel the CUDA fat binary and the MUSA library must be rebuilt with the author's toolchain; this one requires that.

6 The contract with the application

What the application needs is stated as three properties.

(C1) Soundness. A flag is 0 only if the `polDiscTest_cpu` of GetDecics would reject the polynomial.

(C2) Safety. No integer overflow, out-of-bounds access, read of an undefined value, other undefined behaviour, or illegal instruction.

(C3) Integration. The flags reach the exact test as its `polyMask` argument expects, and the 31 test work units give their `.truth` files.

The interface [proved]. After `pdf_init(numP, pSet)`, `pdf_filter(pol, n, flag)` with $n \leq 0$ reads and writes nothing; with $n \geq 1$ it reads only the n rows of 11 coefficients, writes every flag with 0 or 1, and writes 0 only for a polynomial whose leading coefficient is 1 and which does not pass for $S = \{\texttt{pSet}\}$. Before the first `pdf_init`, and after a failed start-up check, every flag is 1; and the module state left by earlier calls changes the routing, never a flag. With Proposition 3 this gives (C1) under (H).

The glue [proved]. The application calls the filter into its flag array and then `polDiscTest_cpu` with the same array as `polyMask`. `polDiscTest_cpu` reads the mask at index k before it writes the flag at index k, touches no other index in that iteration, and writes a flag on every path; so afterwards the flag is 0 where the filter wrote 0 and the exact test's own flag elsewhere, which under (C1) is that flag everywhere: the output of the stock program. Every polynomial of the enumeration is monic (the enumeration sets the leading coefficient to 1 once), and the filter and the exact test get the same S.

Safety [proved]. Overflow: the range proofs; the call counter is kept modulo 64. Bounds: every index is at most n or the lane count, and the chains' indices stay in 0..11 (the masked-loop lemma). Undefined values: padding lanes hold x^{10}, a flag that `chain_general` does not set is set before it is read. Illegal instructions: the AVX2 object runs only when the CPU reports AVX2 and FMA, and contains nothing beyond AVX2. Language-level undefined behaviour: none apart from the conversions and shifts of (S2), (S3), which the start-up check tests.

Named assumptions of the interface. (A1) At most three primes in S: GetDecics' own `int pSet[3]`, and the patch's copy of it, are filled without a bound check (all 31 work units have $|S| = 2$). (A2) One thread: the filter keeps module state, and the application calls it only from the thread that runs the enumeration. (A3) Hypothesis (H) of Proposition 3.

7 The evidence

The theorems are about the program text and the mathematics; the compiler, the device and the code of GetDecics are trusted only as far as the checks below show. All CPU runs on Linux (WSL2, GCC and gfortran 15.2) unless stated.

The compiled filter on complete sets at tiny primes [checked numerically]. The filter was built from the shipped source with its prime search moved to one tiny prime $q \in \{13, 17, 29, 37, 41\}$ (only four constants changed), once in the 16-bit and once in the 32-bit stage, and every polynomial of a set was sent through the AVX2 and the baseline build, each with the normal routing and with every lane forced through `chain_general`: eight runs per polynomial. Every flag and every rejection stage was compared with an exact oracle (Euclid with inverses). The sets: all 13^9 depressed monic polynomials modulo 13 (every monic polynomial mod 13 is a shift of one of them); at 17 all with $a_9 = a_8 = 0$, at $29, 37, 41$ all with $a_9 = \cdots = a_6 = 0$ (which send every chain into `chain_general` and its lockstep branch); and at each prime 200,000 random polynomials with all their shifts and coefficients lifted by large multiples of q. Altogether 25,518,310,785 polynomials and 204,146,486,280 verdicts: none wrong. Both branches of the masked loop ran billions of times.

The arithmetic alone [checked numerically]. The reductions, cut out of the source (not retyped) and compiled with the filter's flags and at `-O0`, against exact models: `mont` on every pair of operands for every odd prime below 2^{13} (333,937,622,331 pairs) and every odd q in $[8193, 8303]$; `sredc` on windows around its limits and random inputs; the loading on 42,352,640 coefficients for 2,585 moduli; the bound propagation of the size limits for every odd q up to 32,767 (small) and 1,100,000,001 (large). With GCC 15.2 (Linux) and 16.2 (MSYS2): no error.

An exact oracle, and deliberate faults [checked numerically]. A test that only feeds polynomials that must pass cannot see a filter that rejects too little [proved]. The oracle test computes the symbol at every filter prime independently and builds its inputs to reach every stage and path of the filter, with metamorphic checks and a test hook that reports the stage of every rejection. Seven deliberate faults of the integer arithmetic, each one changed line, against 1,000,000 polynomials (oracle test total failures, AVX2 build / the old "must pass" test):

fault	oracle test	"must pass" test
none	0	0
zero test $\pm q$ in `chain_block_s`	156	0
zero test $\pm q$ in `chain_general_s`	3,461	3
Euler: $-R$ recognised only as one representative	186,381	0
top piece of a coefficient shifted logically	1,012,353	412,963
wrong loading constants (now caught by the start-up check: filter off)	717,978 missed	0
parity of `chain_general`	236,578	174,075
last stage skipped	7,983	0

The oracle test sees all seven; the floating-point faults (x87, rounding up) give 0 on the integer filter, as Theorem D says, while on the floating-point filter of `42c3270` the oracle test saw 61 false and 304 missed rejections under x87 and 37,329 false under rounding up. Primes moved across the size limits behave as the proofs predict: no wrong verdict inside them; beyond them wrong verdicts appear, at some primes only for directed inputs found by a search. The 146,771 candidates captured from seven real work units pass through the filter with the oracle's verdicts.

The exact test [checked numerically]. Table 1.

The GPU filter [checked numerically]. The kernel's `survives`, compiled for the CPU from the kernel source itself, against the exact oracle prime by prime: 6,000,000 polynomials at the 14 table primes and 3,000,000 at tiny primes and just below the limit 8192, 168,000,000 checks, no false and no missed rejection. On an NVIDIA GTX 1650 (OpenCL, and CUDA with the kernel compiled by NVRTC), the application gave the truth files of 8 work units through OpenCL and 4 through CUDA, before and after the format check was added.

The work units [checked numerically]. The application, built on Linux from GetDecics at `638ca4e` with the patch and the repository's compile lines (BOINC replaced by stand-ins), gave the `.truth` files of all 31 test work units with the AVX2 build and with the baseline build. With `NF_VERIFY=1` (the exact test also on every polynomial, stopping at the first difference), five units in both builds ran 20,008,960 polynomials each without a difference.

Speed [checked numerically]. On the GPU, the device time of all kernel launches, old (floating-point filter) against new kernel, five alternating runs each: median 1217 against 1091 ms and 3715 against 3324 ms on two work units, about 10% less. On the CPU, the application's own timing split over 60 million polynomials, seven alternating runs on a quiet machine: the guard, the start-up check and the other changes of the last revision cost nothing measurable (filter plus exact test, medians: 6.07 → 6.16 s and 6.89 → 6.86 s with AVX2, 7.42 → 7.53 s and 8.26 → 8.16 s baseline, against a spread of up to a third between runs). **Not yet measured**: the CPU integer filter against the floating-point filter of `42c3270` in the application.

8 What it all rests on

The full document ends with a ledger of every claim, how it was checked and what it still assumes; `checks.py` checks its form and every source-line citation of the document. Its contract rows and its named assumptions:

claim	status	rests on
(C1) soundness	[proved]	Theorems A–D, Proposition 3, the interface; assumes (H), compiler correctness, (S2), (S3)
(C2) safety	[proved]	range proofs, size limits, the masked-loop lemma; assumes (S2), (S3) (start-up check), the CPU's report of AVX2/FMA, (A1)
(C3) integration	[proved] [checked numerically]	the glue proposition; 31 units in both builds, 12 on the GPU; assumes (A1), (A2)
GPU filter	[proved]	Theorem E; assumes the device compiler and device correct, host and device code of one format (checked at start-up)
compiler correctness	[named assumption]	tiny-prime test, arithmetic checks with two GCC versions, oracle test, start-up check; docs/BUILDING.md names the checked compilers
(S2), (S3)	[named assumption]	as above; the start-up check switches the filter off if either fails on the running machine
(H)	[named assumption]	no overflow on 733,855 real candidates; GetDecics loses 844,265 of 10^6 large passing polynomials where it fails; the largest value on real candidates is $2^{59.45}$
(A1), (A2)	[named assumption]	all work units have $\lvert S\rvert = 2$; no thread of the program calls the filter

Open items.

- Whether the size limits 8385 and 1,097,010,265 are sharp (no shipped prime is near them).
- (H) for the application's polynomials is measured, not proved; the overflow is a defect of GetDecics, to be reported to its maintainer.
- The GPU filter is checked on the CPU and on one NVIDIA card; AMD, Intel and MUSA devices, and the CUDA kernel built by `nvcc`, were not tested, and no run-time check exercises the filter on the device.
- The start-up check covers its own copy of the coefficient split, not `pdf_filter`'s.
- Only $S = \{2, 5\}$ is exercised by the tests; other sets are covered by the proofs.
- The CPU speed of the integer filter against the floating-point one.

Reproducing. The checks above are scripts of the repository, in `pdfilter_Test` unless stated; [1, Appendix B] gives the commands.

- tiny primes: `tiny_checks.sh`; the arithmetic and the size limits: `arith_checks.sh`;
- the oracle test and the faults: `fault_checks.sh` with `test_pdfilter_oracle.cpp`;
- the exact test: `poldisc_checks.sh`;
- the GPU filter: `test_gpu_filter.cpp`; the mathematics, the citations and the ledger: `checks.py`;
- the work units: `build/test-suite-unix.sh`.

Kept outside the repository: the captured candidates, the Linux build recipe used for the application with BOINC replaced, and the Windows harness of the GPU runs.

References

[1] *Two Euclidean chains for one Legendre symbol: proofs and a worked course for the polDisc pre-filter of Passiflora*, `pdfilter_Test/pdfilter_chains.tex` in the Passiflora repository, 27 October 2026 (the source of every result here, with its proof).

[2] GetDecics, the NumberFields@home application, https://github.com/drivere/get-decics-numberfields (commit `638ca4e`).

[3] H. Cohen, *A Course in Computational Algebraic Number Theory*, GTM 138, Springer, 1993.